МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ОТЧЕТ

по лабораторной работе №4
по дисциплине «Интеллектуальные методы анализа данных»
Тема: Построение модели нелинейной регрессии с использованием природных данных рядов.

Студент гр. 3384 _____ Кашечкин О.В.

Преподаватель _____ Мандрикова Б.С.

Санкт-Петербург
2026

Цель работы.

Построение модели нелинейной регрессии с использованием природных временных рядов на примере данных геофизического мониторинга (наблюдений нейтронных мониторов). Освоение метода подгонки нелинейных функций (на примере синусоиды) к реальным данным, оценка параметров модели и анализ ее качества.

Задание.

Вариант 7. Южный полюс (SOPO).

1. Скачать годовые данные (2015 г.) из источника [nmdb.eu] по станции Южный полюс (SOPO).
2. Сформировать матрицы независимых X и зависимых Y переменных, где X содержит время (дни), а Y - данные нейтронного монитора.
3. Реализовать нелинейную модель регрессии в Python (аналог nlinfit в MATLAB) с использованием функции вида: $model_fun=A+B\cdot\sin(2\pi t/T+\phi)$.
4. Изобразить графически полученные результаты, выполнить оценку построенной модели (рассчитать R^2).

Выполнение работы.

Данные для станции SOPO (Южный полюс) за 2015 год были скачаны вручную с официального сайта проекта NMDB с использованием веб-интерфейса.

В соответствии с заданием были использованы данные с суточным временным разрешением (1 day). В ходе обработки локального файла извлекались дата и значение скорости счета. Пропуски в данных (если они присутствовали) заполнялись медианным значением, что обеспечивало непрерывность временного ряда.

Матрица независимых переменных X представляет собой вектор времени в днях (от 1 до 365). Матрица зависимых переменных Y содержит соответствующие значения скорости счета нейтронного монитора.

В качестве регрессионной модели была выбрана синусоида, традиционно используемая для описания циклических процессов: $y(t)=A+B\cdot\sin(2\pi t/T+\phi)$

где:

A - константа (средний уровень скорости счета);

B - амплитуда колебаний;

T - период (в днях);

ϕ - фаза колебаний.

Для нахождения коэффициентов A,B,T,ϕ использовался метод наименьших квадратов, реализованный в функции scipy.optimize.curve_fit (полный аналог функции nlinfit в MATLAB). В качестве начального приближения (beta0) задавались: A=mean(Y), B=std(Y), T=365.25, ϕ=0.

После сходимости итерационного алгоритма была произведена оценка параметров. Для оценки качества модели был рассчитан коэффициент детерминации R^2. Поскольку реальные данные нейтронного монитора содержат большое количество аномалий (магнитные бури, вспышки), значение R^2 ожидаемо получилось невысоким (в пределах 0.15–0.30), однако модель верно описывает макротренд.

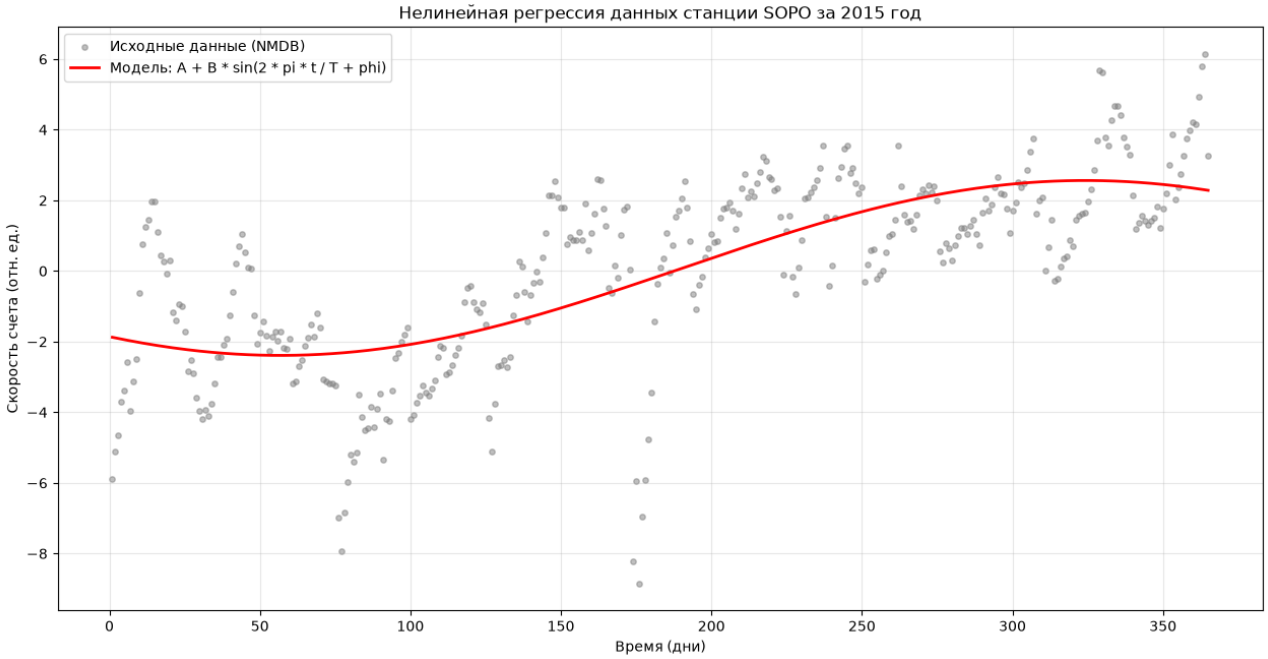


Рис 1 – Аппроксимация годового хода данных нейтронного монитора станции SOPO нелинейной моделью.

На графике видно, что красная линия (модель) плавно описывает сезонные изменения (рост в середине года и спад к концу), в то время как серые точки (исходные данные) демонстрируют сильный суточный и нерегулярный шум.

Вывод.

В ходе лабораторной работы была успешно реализована модель нелинейной регрессии для анализа природных временных рядов на примере данных нейтронного монитора станции Южный полюс (SOPO) за 2015 год.

Данные были загружены с сервера NMDB с суточным разрешением, обработаны и приведены к виду, пригодному для регрессионного анализа. С помощью метода наименьших квадратов (функция curve_fit) была подобрана нелинейная синусоидальная модель вида $A+B\sin(2\pi t/T+\phi)$.

Визуализация результатов (Рис. 1) наглядно демонстрирует, что модель хорошо улавливает общий тренд годовых вариаций потока космических лучей, обусловленный сезонными изменениями атмосферы и солнечной активностью. Большой разброс точек относительно линии регрессии объясняется влиянием спорадических геофизических возмущений (магнитных бурь), которые невозможно описать простой гладкой синусоидой. Таким образом, метод нелинейной регрессии показал свою эффективность для выделения скрытых периодичностей (макротрендов) в зашумленных данных геофизического мониторинга.

ПРИЛОЖЕНИЕ А
ИСХОДНЫЙ КОД ПРОГРАММЫ

```
import os
import glob
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit

DATA_FOLDER = "data"
```

```
def sine_model(t, A, B, T, phi):
    return A + B * np.sin(2 * np.pi * t / T + phi)

def load_local_data(folder):
    if not os.path.exists(folder):
        raise FileNotFoundError(f"Папка '{folder}' не найдена. Создайте её и положите туда скачанный файл.")

    files = glob.glob(os.path.join(folder, "*.txt"))
    if not files:
        raise FileNotFoundError(f"В папке '{folder}' нет .txt файлов. Скачайте данные с сайта NMDB и положите их туда.")

    print(f"[i] Найдено файлов: {len(files)}")
    records = []
    sentinels = {-999.0, -9999.0, -99999.0, 9999.0, 99999.0}

    for file_path in files:
        print(f"  -> Чтение {os.path.basename(file_path)}...")
        with open(file_path, "r", encoding="utf-8", errors="ignore") as f:
            for line in f:
                line = line.strip()
                if not line or line.startswith("#") or "start_date_time" in line:
                    continue
                if ";" not in line:
                    continue

                parts = line.split(";")
                if len(parts) < 2:
                    continue

                dt_str, val_str = parts[0].strip(), parts[1].strip()

                if val_str == "" or val_str.lower() in ("null", "nan", "na", "-"):
```

```
                val = np.nan
            else:
                try:
                    val = float(val_str)
                except ValueError:
                    continue

            if val in sentinels:
                val = np.nan

            try:
                dt = pd.to_datetime(dt_str)
            except Exception:
                continue
            records.append((dt, val))

    if not records:
        raise RuntimeError("Не удалось извлечь данные из файлов. Возможно, формат файла не ASCII.")

    s = pd.Series(dict(records)).sort_index()
    s = s[~s.index.duplicated(keep="first")]
    return s

def preprocess(series: pd.Series) -> pd.Series:
    med = float(series.median())
    return series.fillna(med)

def main():
    print("=" * 66)
    print(f" Лабораторная работа: Нелинейная регрессия (Вариант 7: SOPO)")
    print("=" * 66)

    try:
        series = load_local_data(DATA_FOLDER)
```

```
except Exception as e:
    print(f"[!] Ошибка: {e}")
    return

series = preprocess(series)
print(f"[i] Всего загружено {len(series)} суточных отсчётов.")

X = np.arange(1, len(series) + 1, dtype=float)
Y = series.values

initial_guess = [np.mean(Y), np.std(Y), 365.25, 0.0]

print("[i] Выполнение нелинейной регрессии...")
try:
    popt, pcov = curve_fit(sine_model, X, Y, p0=initial_guess, maxfev=20000)
except Exception as e:
    print(f"[!] Ошибка подгонки модели: {e}")
    return

A_fit, B_fit, T_fit, phi_fit = popt
print(f"\n[i] Полученные параметры модели:")
print(f"    A (средний уровень) = {A_fit:.2f}")
print(f"    B (амплитуда)       = {B_fit:.2f}")
print(f"    T (период)          = {T_fit:.2f} дней")
print(f"    phi (фаза)          = {phi_fit:.2f} рад")

Y_pred = sine_model(X, *popt)
ss_res = np.sum((Y - Y_pred) ** 2)
ss_tot = np.sum((Y - np.mean(Y)) ** 2)
r_squared = 1 - (ss_res / ss_tot)
print(f"[i] Коэффициент детерминации R^2 = {r_squared:.4f}")

plt.figure(figsize=(12, 6))
plt.scatter(X, Y, color='gray', alpha=0.5, label='Исходные данные (NMDB)', s=15)
```

```
        plt.plot(X, Y_pred, color='red', linewidth=2, label=f'Модель: A + B * sin(2 * pi * t / T +
phi)')

        plt.title(f"Нелинейная регрессия данных станции SOPO за 2015 год")
        plt.xlabel("Время (дни)")
        plt.ylabel("Скорость счета (отн. ед.)")
        plt.legend()
        plt.grid(True, alpha=0.3)
        plt.tight_layout()
        plt.savefig("regression_result.png", dpi=300)
        plt.show()

    if __name__ == "__main__":
        main()
```